B2B Multi-Tenant Architecture & Domain Isolation Plan

CodeForge Platform

Engineering Specification

Document 1 of 4

Version	1.0
Status	Draft for Architecture Review
Date	July 2026
Authors	CodeForge Engineering
Distribution	Internal Engineering, Product, Security

CONFIDENTIAL — For internal use only.

Revision History

Version	Date	Author	Description
1.0	2026-07-11	Engineering	Initial specification grounded in existing codebase.

Contents

1 Executive Summary **4**

2 Current Architecture Baseline **4**
2.1 Monorepo Structure 4
2.2 Existing Organization Scoping 5
2.3 Existing Authentication Flow 5
2.4 Existing Compute Architecture 6

3 Multi-Tenant Topology **7**
3.1 Topology Decision: Pooled Compute, Logically Isolated Data 7
3.2 Pooled Compute with Priority Queues 8
3.3 Scaling Strategy 9

4 Tenant Identification and Request Routing **10**
4.1 Domain Resolution Strategy 10
4.2 New Prisma Schema Fields 10
4.3 Tenant Resolution Middleware 11
4.4 Auth Middleware Enhancement 12
4.5 Candidate Token Cross-Tenant Protection 13

5 Database Transition Strategy **14**
5.1 Defence in Depth: RLS on Top of Application Scoping 14
5.2 RLS Implementation 14
5.2.1 Step 1: Application Role and Session Variable 14
5.2.2 Step 2: Enable RLS on Directly Scoped Tables 14
5.2.3 Step 3: Transitive Isolation for Child Tables 15
5.2.4 Step 4: Prisma Client Extension for Session Variable 16
5.3 Migration Sequencing 17
5.4 Worker and Job Tenant Context 17

6 Dedicated Tier for Enterprise Clients **19**
6.1 Architecture 19
6.2 Routing 19
6.3 Cost Considerations 19

7 Candidate-Facing Domain Configuration **20**
7.1 Custom Domain Verification 20
7.2 Email Link Generation 20

8 Backward Compatibility **21**

9 Summary of Required Changes **22**

10 Appendix A: Environment Variable Additions **23**

11 Appendix B: RLS Policy Coverage Matrix **23**

Executive Summary

CodeForge is currently deployed as a self-hosted, single-organization technical assessment platform. Its monorepo (workspaces `@codeforge/api`, `@codeforge/web`, and `@codeforge/shared`) powers a complete hiring pipeline: question authoring, candidate invitation, sandboxed code execution via Judge0, automated grading through BullMQ workers, proctoring, plagiarism detection, and result review.

The existing Prisma schema already scopes every entity to an `Organization` record through direct foreign keys (`orgId` on `User`, `Question`, `Test`) or through transitive relationships (`Invitation` → `Test` → `Organization`). This document specifies the precise modifications required to transition CodeForge from a single-org deployment into a commercially viable, multi-tenant B2B SaaS platform, including:

- Tenant identification and request routing via subdomains and custom apex domains.
- Database-level isolation using PostgreSQL Row Level Security layered on top of the existing `orgId` scoping.
- Compute topology decisions for scaling sandboxed code execution across multiple enterprise clients simultaneously.
- A phased migration strategy that preserves backward compatibility with existing single-org deployments.

Current Architecture Baseline

This section documents the exact state of the codebase that all subsequent proposals build upon. Every file path, model name, and configuration key referenced here exists in the repository at the time of writing.

Monorepo Structure

```
codeforge/
+-- apps/
|   +-- api/                    Express 5 + Prisma + BullMQ
|   |   +-- src/
|   |   |   +-- app.ts          Express app factory
|   |   |   +-- config.ts       Zod-validated env config
|   |   |   +-- worker.ts       Standalone BullMQ worker process
|   |   |   +-- middleware/
|   |   |   |   +-- auth.ts     JWT Bearer verification, RBAC gates
|   |   |   |   +-- candidate.ts  Capability-token auth (x-candidate-token)
```

```
|   |   |   |   +-- rateLimit.ts  express-rate-limit instances
|   |   |   +-- routes/          8 route modules
|   |   |   +-- services/        grading, session, integrity, plagiarism
|   |   |   +-- jobs/            BullMQ queues + processors
|   |   |   +-- lib/             judge0, dockerExecutor, s3, email, tokens
|   |   +-- prisma/
|   |       +-- schema.prisma    430-line schema (17 models, 7 enums)
|   |       +-- seed.ts          Bootstrap admin + demo data
|   +-- web/                     Next.js 15 + Tailwind
|       +-- src/app/
|           +-- (dash)/          Staff dashboard (recruiter/admin routes)
|           +-- t/               Candidate test-taking UI
|           +-- login/           Staff authentication
+-- packages/
|   +-- shared/                  Zod schemas, type unions, scoring logic
+-- docker-compose.yml           Postgres, Redis, MinIO, Judge0 (2 networks)
+-- judge0.conf                  Execution limits, network=false
+-- SECURITY.md                  Isolation model documentation
```

Existing Organization Scoping

The Prisma schema already contains an `Organization` model and `orgId` foreign keys on every staff-facing entity:

```prisma
model Organization {
  id             String   @id @default(cuid())
  name           String
  brandingConfig Json     @default("{}")
  createdAt      DateTime @default(now())

  users     User[]
  questions Question[]
  tests     Test[]
}
```

The `User`, `Question`, and `Test` models each carry `orgId` with `@@index([orgId])` or `@@index([orgId, ...])` compound indices. Downstream entities (`Invitation`, `TestSession`, `Submission`, `JudgeResult`, etc.) are transitively scoped through their parent `Test` or `TestSession`.

Existing Authentication Flow

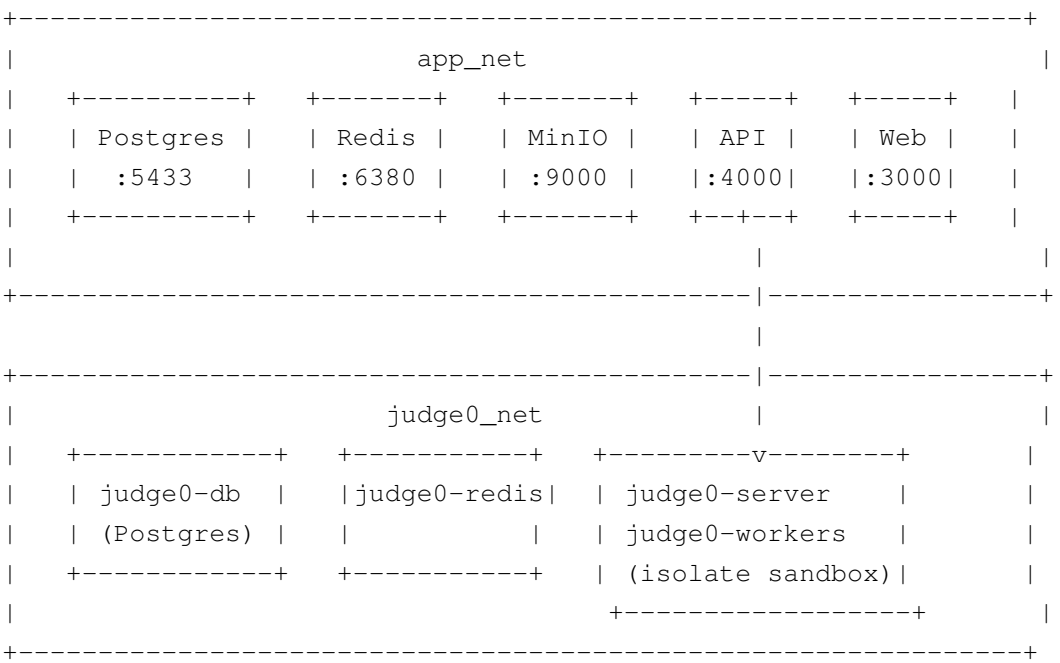
Staff authentication (`middleware/auth.ts`) extracts a JWT Bearer token containing an `AccessTokenPayload`:

```typescript
export interface AccessTokenPayload {
  sub: string;     // User.id
  orgId: string;   // Organization.id
  role: Role;      // super_admin | recruiter | reviewer
  email: string;
  name: string;
}
```

The `orgId` is embedded in the token at login time and attached to `req.user`. Route handlers currently enforce org scoping manually by filtering Prisma queries on `req.user.orgId`. This is correct but relies on every query author remembering to include the filter.

Candidate authentication (`middleware/candidate.ts`) uses an opaque 256-bit capability token sent as the `x-candidate-token` header. The middleware loads the `Invitation` (with its related `Test`) and attaches it to `req.candidate`. Org scoping is implicit: the token is bound to exactly one `Invitation` → `Test` → `Organization`.

Existing Compute Architecture



The API and Worker containers are the only processes that bridge `app_net` and `judge0_net`. Judge0 runs with `ENABLE_NETWORK=false` and `ALLOW_ENABLE_NETWORK=false`. The Docker executor fallback applies `-network none`, memory caps, CPU limits, and PID limits.

Multi-Tenant Topology

Topology Decision: Pooled Compute, Logically Isolated Data

For a code-execution platform, three multi-tenant topologies are viable:

Table 1: Multi-Tenant Topology Comparison

Dimension	**Full Silo**	**Hybrid (Bridge)**	**Pooled**
Database	One DB per tenant	One DB, schemas per tenant	One DB, RLS per tenant
Compute	Dedicated Judge0 cluster per tenant	Shared Judge0, per-tenant priority queues	Shared Judge0, global queue
Cost at 200 tenants	Prohibitive	Moderate	Lowest
Isolation strength	Strongest	Strong	Strong (with RLS)
Ops complexity	Highest	Medium	Lowest
Time to onboard	Minutes (provision infra)	Seconds (create schema)	Seconds (insert row)

Recommendation: Pooled with Priority-Weighted Queues. The existing architecture already uses a single Postgres instance and a single BullMQ/Redis instance. Moving to full silo would require provisioning and managing separate infrastructure per client, which is operationally untenable below approximately 50 paying enterprise clients. The pooled model, hardened with PostgreSQL Row Level Security and per-tenant BullMQ priority queues, delivers strong isolation at minimal operational cost.

For enterprise clients who contractually require physical isolation (SOC 2 Type II, HIPAA-adjacent requirements), a "Dedicated Tier" can be offered where their Judge0 worker pool and database schema are physically separated. This is documented in Section 6.

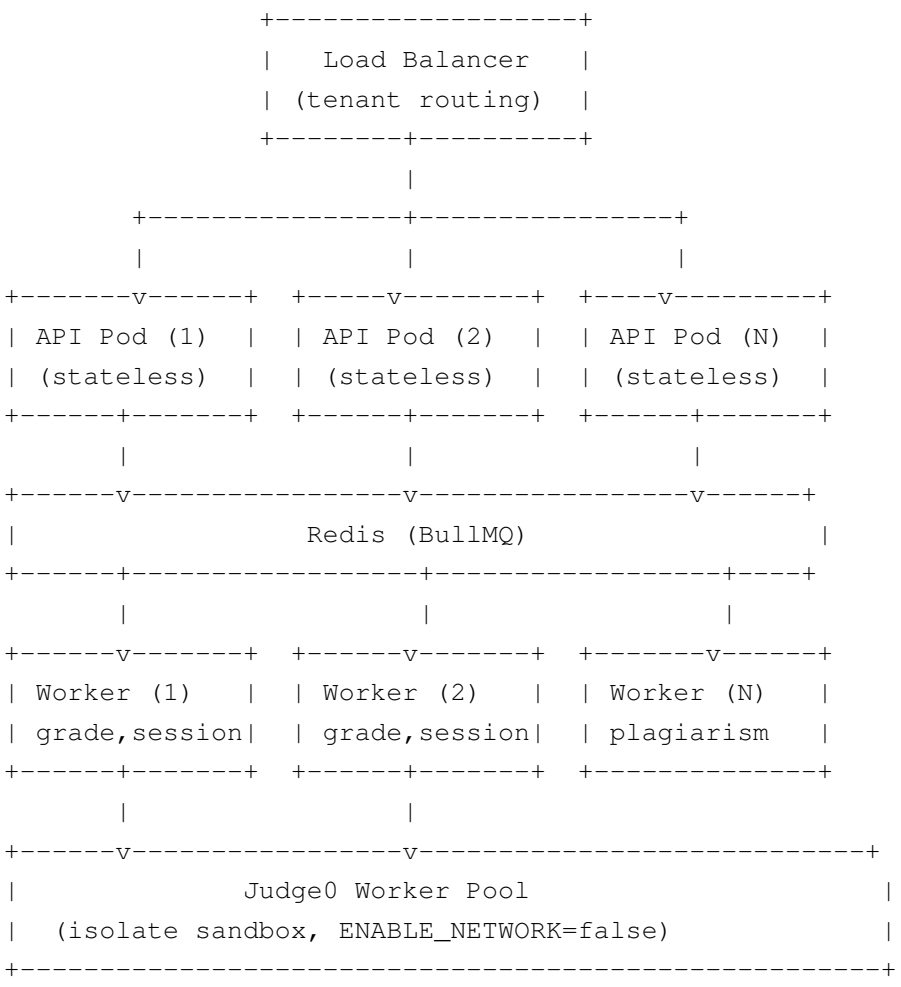
Pooled Compute with Priority Queues

The existing BullMQ queues (`grade`, `session`, `plagiarism`) currently process jobs in FIFO order with no tenant awareness. The transition introduces **tenant-weighted priority** so that a burst of submissions from Tenant A does not starve Tenant B.

```
// jobs/queues.ts -- modified enqueueGradeSubmission
export async function enqueueGradeSubmission(
  submissionId: string,
  orgId: string,
  tier: "standard" | "priority" | "dedicated"
): Promise<void> {
  const priority = tier === "priority" ? 2 : tier === "dedicated"
     ? 1 : 5;
  await gradeQueue.add(
    "grade-submission",
    { submissionId, orgId },
    { priority }
  );
}
```

BullMQ's built-in priority system (lower number = higher priority) ensures that enterprise-tier clients get their submissions graded ahead of standard-tier clients during contention periods, without requiring separate queues or workers.

Scaling Strategy



The API layer (`apps/api/src/app.ts`) is already stateless: JWT authentication is self-contained, and all mutable state lives in Postgres or Redis. Horizontal scaling requires only adding more API pods behind the load balancer and more BullMQ workers subscribing to the same Redis queues.

Judge0 workers scale independently. The `COUNT_OF_WORKERS` setting in `judge0.conf` (currently `2`) controls parallelism per Judge0 container. Additional `judge0-workers` containers can be added to the `judge0_net` network to increase throughput.

Tenant Identification and Request Routing

Domain Resolution Strategy

Every inbound HTTP request must be attributed to exactly one tenant. CodeForge supports two domain patterns:

Table 2: Domain Patterns and Resolution Logic

Pattern	Example	Resolution
Platform subdomain	`acme.codeforge.com`	Extract subdomain prefix, look up `Organization.slug`
Custom apex domain	`assessments.acme.com`	Look up `Organization.customDomain`

New Prisma Schema Fields

The `Organization` model requires three new fields to support domain-based tenant identification:

```
model Organization {
  id             String   @id @default(cuid())
  name           String
  // NEW: URL-safe identifier for subdomain routing
  slug           String   @unique
  // NEW: Optional custom domain (e.g., "assessments.acme.com")
  customDomain   String?  @unique
  // NEW: Verified flag -- custom domain only activates after
  // DNS TXT verification confirms ownership.
  domainVerified Boolean  @default(false)

  brandingConfig Json     @default("{}")
  createdAt      DateTime @default(now())

  users     User[]
  questions Question[]
  tests     Test[]
}
```

Tenant Resolution Middleware

A new middleware, `middleware/tenant.ts`, runs before `auth.ts` on every request. It resolves the tenant from the request hostname and attaches it to the Express request context.

```typescript
// middleware/tenant.ts
import type { NextFunction, Request, Response } from "express";
import { prisma } from "../lib/prisma";
import { config } from "../config";

export interface TenantContext {
  orgId: string;
  slug: string;
  customDomain: string | null;
}

declare module "express-serve-static-core" {
  interface Request {
    tenant?: TenantContext;
  }
}

const PLATFORM_APEX = config.platformDomain; // "codeforge.com"
const tenantCache = new Map<string, TenantContext>();
const CACHE_TTL_MS = 5 * 60 * 1000;

export async function resolveTenant(
  req: Request, _res: Response, next: NextFunction
) {
  const hostname = req.hostname.toLowerCase();

  // 1. Check cache
  const cached = tenantCache.get(hostname);
  if (cached) { req.tenant = cached; return next(); }

  let org: TenantContext | null = null;

  // 2. Platform subdomain: <slug>.codeforge.com
  if (hostname.endsWith(`.${PLATFORM_APEX}`)) {
    const slug = hostname.replace(`.${PLATFORM_APEX}`, "");
    if (slug && !slug.includes(".")) {
      const row = await prisma.organization.findUnique({
```

```
        where: { slug },
        select: { id: true, slug: true, customDomain: true },
      });
      if (row) org = { orgId: row.id, slug: row.slug,
                       customDomain: row.customDomain };
    }
  }

  // 3. Custom domain: assessments.acme.com
  if (!org) {
    const row = await prisma.organization.findFirst({
      where: { customDomain: hostname, domainVerified: true },
      select: { id: true, slug: true, customDomain: true },
    });
    if (row) org = { orgId: row.id, slug: row.slug,
                     customDomain: row.customDomain };
  }

  if (!org) return next(new Error("Unknown tenant"));

  tenantCache.set(hostname, org);
  setTimeout(() => tenantCache.delete(hostname), CACHE_TTL_MS);

  req.tenant = org;
  next();
}
```

Auth Middleware Enhancement

The existing `requireAuth` middleware in `middleware/auth.ts` currently trusts the `orgId` embedded in the JWT without cross-referencing the request's tenant context. In a multi-tenant deployment, a stolen or reused token from one subdomain must not grant access under a different subdomain. The fix is a three-line addition:

```
// Inside requireAuth, after verifying the token:
if (req.tenant && payload.orgId !== req.tenant.orgId) {
  return next(forbidden(
    "Token organization does not match the requested tenant"
  ));
}
```

Candidate Token Cross-Tenant Protection

The `requireCandidate` middleware loads the `Invitation` → `Test` chain. The tenant cross-check is:

```
// Inside requireCandidate, after loading the invitation:
if (req.tenant && invitation.test.orgId !== req.tenant.orgId) {
  return next(unauthorized("This test link is not valid"));
}
```

This prevents a candidate token issued by Tenant A from being used on Tenant B's subdomain.

Database Transition Strategy

Defence in Depth: RLS on Top of Application Scoping

The existing codebase enforces org scoping at the application layer: every Prisma query in route handlers includes a `where: { orgId: req.user.orgId }` clause. This is necessary but insufficient for a B2B platform. A single missed filter, a new route added without the clause, or a raw SQL query in a migration script could leak data across tenants.

PostgreSQL Row Level Security provides a database-level safety net. Even if the application layer fails to filter by `orgId`, the database itself refuses to return rows belonging to other tenants.

RLS Implementation

Step 1: Application Role and Session Variable

Create a non-superuser role that the application connects as, and use a session variable to communicate the current tenant:

```
-- Migration: 001_rls_setup.sql
CREATE ROLE codeforge_app LOGIN PASSWORD '...';
GRANT USAGE ON SCHEMA public TO codeforge_app;
GRANT SELECT, INSERT, UPDATE, DELETE
  ON ALL TABLES IN SCHEMA public TO codeforge_app;

-- The session variable that RLS policies will read.
-- Set per-connection by the application before any query.
ALTER DATABASE codeforge SET app.current_org_id = '';
```

Step 2: Enable RLS on Directly Scoped Tables

The three tables with a direct `orgId` column receive RLS policies first:

```
-- organizations: any tenant can only see its own row
ALTER TABLE "Organization" ENABLE ROW LEVEL SECURITY;
ALTER TABLE "Organization" FORCE ROW LEVEL SECURITY;
CREATE POLICY org_isolation ON "Organization"
  FOR ALL
  USING (id = current_setting('app.current_org_id')::text);

-- users: scoped by orgId
ALTER TABLE "User" ENABLE ROW LEVEL SECURITY;
ALTER TABLE "User" FORCE ROW LEVEL SECURITY;
```

```
CREATE POLICY user_isolation ON "User"
  FOR ALL
  USING ("orgId" = current_setting('app.current_org_id')::text);

-- questions: scoped by orgId
ALTER TABLE "Question" ENABLE ROW LEVEL SECURITY;
ALTER TABLE "Question" FORCE ROW LEVEL SECURITY;
CREATE POLICY question_isolation ON "Question"
  FOR ALL
  USING ("orgId" = current_setting('app.current_org_id')::text);

-- tests: scoped by orgId
ALTER TABLE "Test" ENABLE ROW LEVEL SECURITY;
ALTER TABLE "Test" FORCE ROW LEVEL SECURITY;
CREATE POLICY test_isolation ON "Test"
  FOR ALL
  USING ("orgId" = current_setting('app.current_org_id')::text);
```

Step 3: Transitive Isolation for Child Tables

Tables without a direct `orgId` (e.g., `Invitation`, `TestSession`, `Submission`) are isolated by joining to their parent:

```
ALTER TABLE "Invitation" ENABLE ROW LEVEL SECURITY;
ALTER TABLE "Invitation" FORCE ROW LEVEL SECURITY;
CREATE POLICY invitation_isolation ON "Invitation"
  FOR ALL
  USING (
    "testId" IN (
      SELECT id FROM "Test"
      WHERE "orgId" = current_setting('app.current_org_id')::text
    )
  );
```

The same pattern applies to `TestSession` (via `testId`), `Submission` (via `sessionId` → `TestSession.testId`), and all downstream models.

Step 4: Prisma Client Extension for Session Variable

The Prisma client must set `app.current_org_id` at the start of every database session. A Prisma client extension handles this transparently:

```
// lib/prisma.ts -- modified
import { PrismaClient } from "@prisma/client";

const basePrisma = new PrismaClient();

export function tenantPrisma(orgId: string) {
  return basePrisma.$extends({
    query: {
      $allOperations: async ({ args, query }) => {
        await basePrisma.$executeRawUnsafe(
          `SET LOCAL app.current_org_id = '${orgId}'`
        );
        return query(args);
      },
    },
  });
}

// For migration scripts and seed operations that need
// unrestricted access, the base client is still available.
export { basePrisma as prisma };
```

Route handlers then use the tenant-scoped client:

```
// Example: question.routes.ts
router.get("/", requireAuth, async (req, res) => {
  const db = tenantPrisma(req.user!.orgId);
  const questions = await db.question.findMany({
    where: { isArchived: false },
    // orgId filter is still here for defense-in-depth,
    // but RLS enforces it even if this line is removed.
    // orgId: req.user!.orgId,
  });
  res.json(questions);
});
```

Migration Sequencing

The transition must be performed without downtime and without breaking the existing single-org deployment:

Table 3: Database Migration Phases

Phase	Migration	Description
M1	Schema addition	Add `slug`, `customDomain`, `domainVerified` columns to `Organization`. Backfill `slug` from a slugified `name`.
M2	RLS setup	Create the `codeforge_app` role, enable RLS, create policies. Run in a transaction with the application temporarily using superuser credentials for the migration.
M3	Connection swap	Switch `DATABASE_URL` from the superuser role to `codeforge_app`. Deploy the Prisma client extension.
M4	Validation	Run the full test suite against the RLS-enabled database. Verify that cross-tenant queries return zero rows.

Worker and Job Tenant Context

BullMQ jobs are currently tenant-unaware. The `gradeSubmission` function in `services/grading.ts` loads a `Submission` with its related `Question` and `TestSession`. Since the worker process does not receive HTTP requests (and therefore has no tenant middleware), it must set the RLS session variable explicitly:

```
// services/grading.ts -- modified
export async function gradeSubmission(
  submissionId: string
): Promise<void> {
  // Load with superuser client to discover the orgId
  const sub = await prisma.submission.findUnique({
    where: { id: submissionId },
    include: {
      question: true,
      session: { include: { test: true } },
    },
  });
  if (!sub) return;

  const orgId = sub.session.test.orgId;
  const db = tenantPrisma(orgId);
  // All subsequent operations use the tenant-scoped client
```

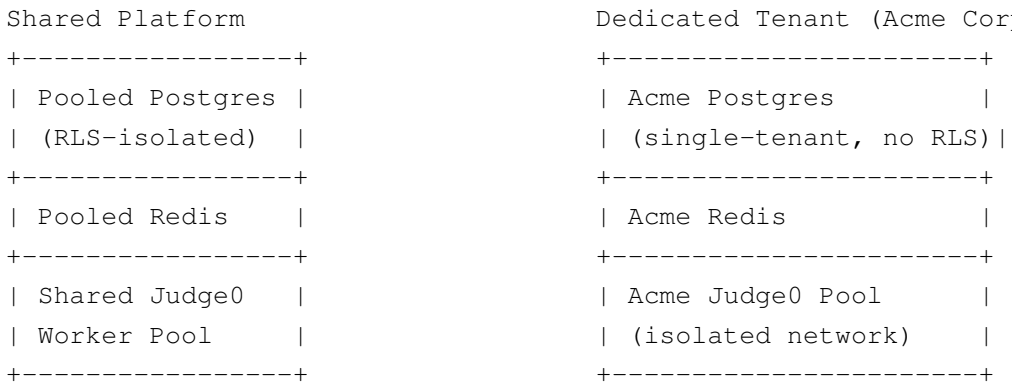
```
    // ...
}
```

Alternatively, the `orgId` can be included in the BullMQ job payload at enqueue time (as shown in Section 3.2), eliminating the need for a superuser lookup.

Dedicated Tier for Enterprise Clients

Clients with contractual requirements for physical data isolation can be offered a "Dedicated Tier" that runs on separate infrastructure while sharing the same codebase and deployment pipeline.

Architecture



Routing

The tenant resolution middleware checks a `tier` field on the `Organization` record. For dedicated-tier tenants, the API process switches its database connection string to the tenant-specific Postgres instance. This is implemented via a connection pool registry keyed by `orgId`.

Cost Considerations

Dedicated infrastructure is provisioned through Terraform/Pulumi modules and billed at a premium. The platform control plane manages lifecycle operations (provisioning, scaling, backup, teardown) through an internal admin API that is not exposed to tenant administrators.

Candidate-Facing Domain Configuration

Candidates access assessments through links embedded in invitation emails. The current link format is:

```
http://localhost:3000/t/<token>
```

In the multi-tenant deployment, candidate links must route through the tenant's domain:

```
https://acme.codeforge.com/t/<token>
-- or --
https://assessments.acme.com/t/<token>
```

Custom Domain Verification

When a tenant administrator configures a custom domain, CodeForge generates a DNS TXT verification token:

```
_codeforge-verification.assessments.acme.com  TXT  "cf-verify=abc123..."
```

A background job periodically checks DNS for this record. Once verified, `domainVerified` is set to `true` and the domain becomes active. TLS certificates are provisioned automatically through a wildcard certificate for `*.codeforge.com` (platform subdomains) and Let's Encrypt / ACME for custom apex domains, managed by the load balancer or a reverse proxy such as Caddy.

Email Link Generation

The existing `sendInvitationEmail` function in `lib/email.ts` uses `config.webOrigin` to construct the link. In multi-tenant mode, the link origin is derived from the tenant's configuration:

```
function candidateOrigin(org: Organization): string {
  if (org.customDomain && org.domainVerified) {
    return `https://${org.customDomain}`;
  }
  return `https://${org.slug}.${config.platformDomain}`;
}
```

Backward Compatibility

The migration must not break existing single-org self-hosted deployments. The following guarantees are maintained:

1. **No tenant middleware required.** If the `PLATFORM_DOMAIN` environment variable is unset, the tenant resolution middleware is bypassed, and the system operates in single-org mode exactly as it does today.

2. **RLS is opt-in.** The RLS migration is a separate, numbered Prisma migration. Self-hosted operators who do not run it continue to operate without RLS. The Prisma client extension gracefully skips `SET LOCAL` when no `orgId` is provided.

3. **Seed script compatibility.** The existing seed script (`prisma/seed.ts`) creates one `Organization` and one `User`. The `slug` field is backfilled from `name` with a default value of `"default"`.

4. **Environment variable additions are optional.** New variables (`PLATFORM_DOMAIN`, `DEDICATED_TIER_DB_URLS`) default to empty strings, preserving the current behaviour.

Summary of Required Changes

Table 4: Change Inventory for Multi-Tenant Transition

File / Area	Change Type	Description
`schema.prisma`	Modify	Add `slug`, `customDomain`, `domainVerified` to Organization
`middleware/tenant.ts`	New file	Tenant resolution from hostname
`middleware/auth.ts`	Modify	Cross-check `orgId` against `req.tenant`
`middleware/candidate.ts`	Modify	Cross-check `test.orgId` against `req.tenant`
`lib/prisma.ts`	Modify	Add `tenantPrisma()` extension
`config.ts`	Modify	Add `PLATFORM_DOMAIN` to Zod schema
`app.ts`	Modify	Mount `resolveTenant` before route handlers
`jobs/queues.ts`	Modify	Accept `orgId` and `tier` in job payloads; set priority
`services/grading.ts`	Modify	Use `tenantPrisma(orgId)` for RLS compliance
`lib/email.ts`	Modify	Derive candidate link origin from tenant config
SQL migration	New file	RLS policies for all 17 tables
`docker-compose.yml`	Modify	Optional dedicated-tier service definitions

Appendix A: Environment Variable Additions

Table 5: New Environment Variables

Variable	Default	Purpose
`PLATFORM_DOMAIN`	`""`	Base domain for tenant subdomains (e.g., `codeforge.com`). Empty disables multi-tenant routing.
`RLS_ENABLED`	`"false"`	When `"true"`, the Prisma client extension sets `app.current_org_id` on every connection.
`TENANT_CACHE_TTL_MS`	`"300000"`	Duration (ms) to cache tenant resolution results in memory.

Appendix B: RLS Policy Coverage Matrix

Model	Isolation Column	Policy Type
Organization	`id` (direct)	Direct match
User	`orgId` (direct)	Direct match
RefreshToken	via `User.orgId`	JOIN to User
Question	`orgId` (direct)	Direct match
TestCase	via `Question.orgId`	JOIN to Question
Test	`orgId` (direct)	Direct match
TestQuestion	via `Test.orgId`	JOIN to Test
QuestionPool	via `Test.orgId`	JOIN to Test
Invitation	via `Test.orgId`	JOIN to Test
TestSession	via `Test.orgId`	JOIN to Test
AnswerDraft	via `TestSession`	2-hop JOIN
Submission	via `TestSession`	2-hop JOIN
JudgeResult	via `Submission`	3-hop JOIN
ProctoringEvent	via `TestSession`	2-hop JOIN
PlagiarismReport	via `Submission`	3-hop JOIN
EditorEventBatch	via `TestSession`	2-hop JOIN
ReviewNote	via `TestSession`	2-hop JOIN
VoiceLog	(N/A, Udyox-only model)	N/A

End of Document 1. Continues in Document 2: Zero-Trust Compute Sandbox & Multi-Tenant Security Spec.